INFORMATIONSBLATT

# IB02 – Erste PHP-Befehle: echo, Kommentare und date()

Dynamische Webseiten mit PHP · LS01 | 2BKI2 · Berufskolleg Technische Assistenz · Schuljahr 2026/2027

**🎯 Darum geht es**

Sie schreiben PHP-Blöcke in eine HTML-Seite, geben mit `echo` Text und HTML aus, kommentieren Ihren Code, lassen den Server Datum und Uhrzeit einsetzen und lesen Fehlermeldungen.

## 1 | Der PHP-Block

PHP-Code steht in einer ganz normalen HTML-Seite – eingerahmt von `<?php` und `?>`. Alles zwischen diesen beiden Markierungen führt der Server aus, alles außerhalb bleibt HTML.

**Ein PHP-Block mitten im HTML** index.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>Mein erstes PHP</title></head>
<body>

<h1>Eiscafé Dolce Vita</h1>
<?php
  echo "Diese Zeile schreibt der Server.";
?>
<p>Diese Zeile steht fest im HTML.</p>

</body>
</html>
```

**👁 SO SIEHT ES IM BROWSER AUS**

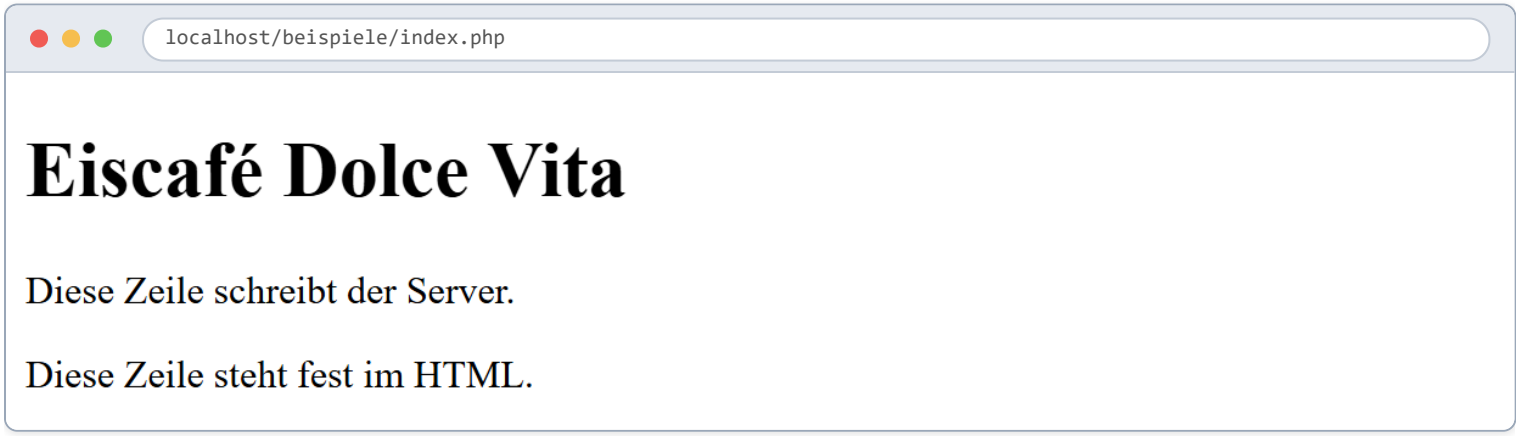



Abbildung 1: Der PHP-Block wird durch seine Ausgabe ersetzt. Im Browser ist kein Unterschied zu erkennen.

Im Seitenquelltext kommt davon beim Browser nur das an:

**Seitenquelltext im Browser (Strg + U)**

```
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>Mein erstes PHP</title></head>
<body>

<h1>Eiscafé Dolce Vita</h1>
Diese Zeile schreibt der Server.<p>Diese Zeile steht fest im HTML.</p>

</body>
</html>
```

📌 **DREI REGELN**

- Die Datei muss auf `.php` enden.
- Jede Anweisung endet mit einem **Semikolon** `;`.
- Eine Seite darf beliebig viele PHP-Blöcke enthalten.

## 2 | Ausgeben mit echo

`echo` schreibt einen Text in die Seite. Der Text steht in Anführungszeichen. Weil PHP HTML *erzeugt*, dürfen im Text auch HTML-Tags stehen – der Browser behandelt sie wie jedes andere HTML.

**HTML mit echo erzeugen** — beispiel.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>echo</title></head>
<body>

<?php
  echo "<h2>Unsere Klassiker</h2>";
  echo "<p>Vanille, Schokolade und <strong>Pistazie</strong>.</p>";
  echo "<p>Heute neu: Mango mit Minze</p>";
?>

</body>
</html>
```

👁 **SO SIEHT ES IM BROWSER AUS**

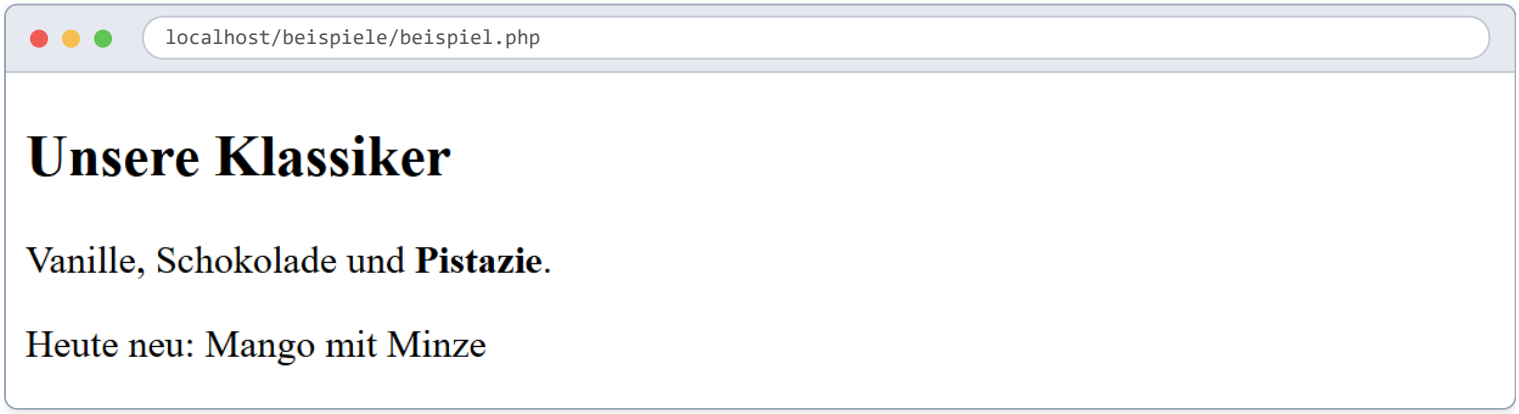



Abbildung 2: Mit echo ausgegebene Tags werden vom Browser ganz normal dargestellt.

> ⚠ **GERADE ANFÜHRUNGSZEICHEN**
>
> PHP versteht nur die geraden Zeichen `"` und `'`. Typografische Zeichen wie „ “ aus Word oder aus E-Mails führen zu einer Fehlermeldung. Code deshalb immer im Editor schreiben, nicht in Word.

## 3 | Kommentare

Kommentare sind Notizen für Menschen. PHP überspringt sie. Anders als HTML-Kommentare verlassen PHP-Kommentare nie den Server.

**Drei Arten von PHP-Kommentaren** — beispiel.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>Kommentare</title></head>
<body>

<!-- HTML-Kommentar: kommt beim Browser an -->
<?php
  // PHP-Kommentar: bleibt auf dem Server
  # Diese Schreibweise geht auch
  /* Mehrzeiliger Kommentar:
     Preise bitte jeden März prüfen! */
  echo "<p>Frisch gemacht seit 1998.</p>";
?>

</body>
</html>
```

**👁 SO SIEHT ES IM BROWSER AUS**

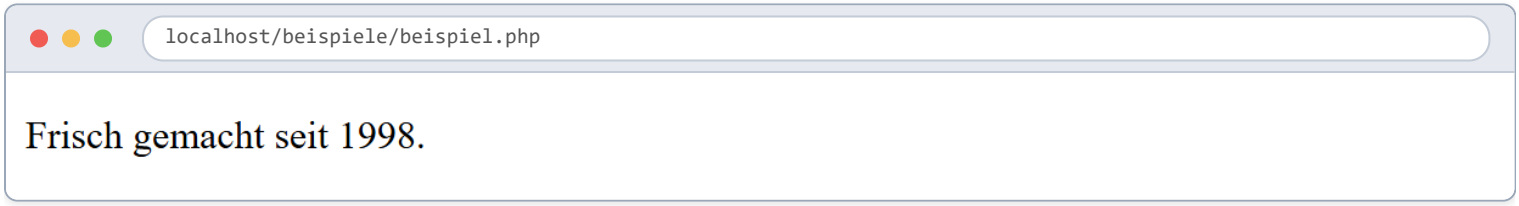



Abbildung 3: Nur der Absatz wird angezeigt.

**Seitenquelltext im Browser (Strg + U)**

```
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>Kommentare</title></head>
<body>

<!-- HTML-Kommentar: kommt beim Browser an -->
<p>Frisch gemacht seit 1998.</p>
</body>
</html>
```

| Schreibweise | Wirkung |
|---|---|
| `// Text` oder `# Text` | Kommentar bis zum Zeilenende |
| `/* … */` | Kommentar über mehrere Zeilen |
| `<!-- … -->` | HTML-Kommentar: für Besucher im Seitenquelltext **sichtbar** |

## 4 | Datum und Uhrzeit mit date()

`date()` ist eine **Funktion**: ein fertiger Befehl, der etwas ausrechnet und das Ergebnis zurückgibt. In den Klammern steht ein **Formatmuster**. Jeder Buchstabe darin wird durch einen Teil des aktuellen Datums ersetzt, alle anderen Zeichen bleiben stehen.

**Datum, Uhrzeit, Jahr** — beispiel.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>Datum</title></head>
<body>

<p>Heute ist der <?php echo date("d.m.Y"); ?>.</p>
<p>Es ist <?php echo date("H:i"); ?> Uhr.</p>
<p>Wir haben das Jahr <?= date("Y") ?>.</p>

</body>
</html>
```

**SO SIEHT ES IM BROWSER AUS**

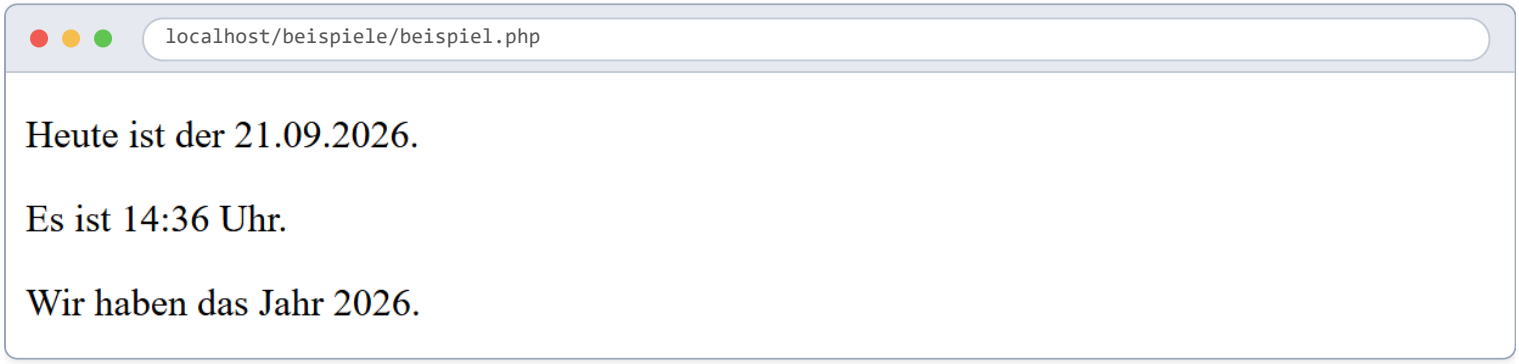



Abbildung 4: Datum und Uhrzeit zum Zeitpunkt des Aufrufs.

| Zeichen | Bedeutung | Beispiel |
|---|---|---|
| `d` | Tag, zweistellig | 07 |
| `j` | Tag ohne führende Null | 7 |
| `m` | Monat, zweistellig | 09 |
| `n` | Monat ohne führende Null | 9 |
| `Y` | Jahr, vierstellig | 2026 |
| `y` | Jahr, zweistellig | 26 |
| `H` | Stunde (00–23) | 14 |
| `i` | Minute (00–59) | 05 |
| `N` | Wochentag als Zahl (1 = Montag … 7 = Sonntag) | 5 |

> 💡 **DIE KURZFORM <?= ?>**
>
> Für eine einzelne Ausgabe mitten im HTML gibt es eine Abkürzung: `<?= date("Y") ?>` bedeutet dasselbe wie `<?php echo date("Y"); ?>`. Beide Schreibweisen sind richtig.

> ⚠️ **FALSCHE UHRZEIT?**
>
> PHP rechnet mit der Zeitzone, die in seiner Konfigurationsdatei `php.ini` steht. Liegt die Uhrzeit ein bis zwei Stunden daneben, fehlt dort `date.timezone=Europe/Berlin`. Die Lehrkraft kann das einmalig einstellen.

## 5 | Fehlermeldungen lesen

Fehler gehören zum Programmieren dazu. PHP bricht dann ab und schreibt eine Meldung in die Seite. Hier fehlt in Zeile 7 das Semikolon:

**Fehlendes Semikolon** — beispiel.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>Fehler</title></head>
<body>

<?php
  echo "Willkommen"
  echo " im Eiscafé!";
?>

</body>
</html>
```

**👁 SO SIEHT ES IM BROWSER AUS**

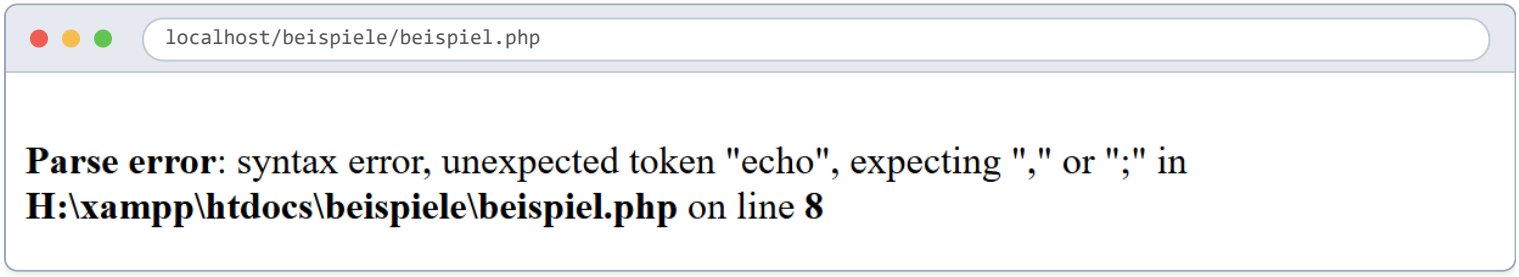



Abbildung 5: Eine typische Fehlermeldung: Art, Beschreibung, Datei und Zeile.

| Teil der Meldung | Bedeutung |
|---|---|
| **Parse error** | PHP konnte den Code gar nicht erst lesen – ein Schreibfehler. Die ganze Datei wird nicht ausgeführt. |
| **Fatal error** | Der Code war lesbar, aber beim Ausführen ging etwas schief, z. B. ein unbekannter Befehl. |
| *unexpected token "echo"* | An dieser Stelle hat PHP etwas anderes erwartet – hier ein `;`. |
| *on line 8* | Hier hat PHP den Fehler **bemerkt**. |

✅ **FAUSTREGEL**

Die gemeldete Zeile ist die Stelle, an der PHP *stolpert*. Die Ursache liegt oft **eine Zeile davor** – wie beim fehlenden Semikolon oben. Also immer die gemeldete Zeile *und* die Zeile darüber prüfen.

---